6. Iterator in Java — Quick Notes

Iterator

Purpose

Used to:

- Traverse collections
- Remove items while traversing

Iterable vs Iterator

Interface	Purpose
Iterable	Provides iterator()
Iterator	Used for traversal

Iterator<Integer> it = c.iterator();

Important Iterator Methods

Method	Purpose
hasNext()	Checks next element exists
next()	Returns next element
remove()	Removes current element

Basic Example

```
Collection<Integer> c = new ArrayList<>();

c.add(10);
c.add(15);
c.add(20);

Iterator<Integer> it = c.iterator();

while(it.hasNext()) {
   System.out.println(it.next());
}
```

Output:

```
10
15
20
```

Important Concept

Iterator initially points: BEFORE first element

Example: | 10 | 15 | 20 |

iterator

hasNext()

Checks: Is there an element after iterator?

Example: it.hasNext()

Returns:

- `true` → next element exists
- `false` → traversal complete

next()

Does TWO things

1. Returns next element
2. Moves iterator forward

Example: int x = it.next();

If iterator before: 10

Then:

- returns `10`
- moves iterator forward

remove()

Important

Removes: element returned by previous next()

Example: it.remove();

Remove Even Numbers Example

```
Collection<Integer> c = new ArrayList<>();

c.add(10);
c.add(15);
c.add(20);

Iterator<Integer> it = c.iterator();
```

```
while(it.hasNext()) {

   int x = it.next();

   if(x % 2 == 0)
      it.remove();
}

System.out.println(c);
```

Output: [15]

Why Iterator is Powerful?

Same code works for:

- ArrayList
- LinkedList
- HashSet
- TreeSet

Because all implement: Iterable

Generic Iterator Example

void removeByCondition(Collection<?> c)

Can work with:

- Any collection
- Any datatype

Important Rule

❌ Wrong:

```
it.remove();
it.remove();
```

after one `next()` call.

✅ Correct: next() → remove()

Only ONE remove allowed after each next().

Otherwise:

- Undefined behavior / exception